

Java Access Specifiers

Access Specifiers (Access Modifiers) define the visibility (accessibility) of classes, variables, methods, and constructors in Java. They determine who can access a member from where.

Java provides four access specifiers:

1. **public**
2. **protected**
3. **default (package-private)** (*no keyword*)
4. **private**

Why Do We Need Access Specifiers?

Access specifiers help to:

- Control access to class members.
- Protect sensitive data (Encapsulation).
- Improve code security.
- Prevent unauthorized access.
- Organize code into packages.

Types of Access Specifiers

Access Specifier	Same Class	Same Package	Subclass (Different Package)	Other Package
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default (<i>no keyword</i>)	✓	✓	✗	✗
private	✓	✗	✗	✗

Public Access Specifier

A public member can be accessed from anywhere in the program

```
class Student {
    public String name = "Ashok";
    public void display() {
        System.out.println(name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.name);
        s.display();
    }
}
```

Ashok
Ashok

Private Access Specifier

A private member can be accessed only within the same class.

```
class Student {  
    private int marks = 95;  
    private void showMarks() {  
        System.out.println(marks);  
    }  
    public void display() {  
        showMarks();  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
        // System.out.println(s.marks);    // Error  
        // s.showMarks();                  // Error  
        s.display();  
    }  
}
```

95

Default (Package-Private) Access Specifier

If no access specifier is specified, Java uses the default (package-private) access level.

Members are accessible only within the same package.

```
class Student {  
    int age = 20; // Default (package-private), NOT private  
    void display() {  
        System.out.println(age);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
        System.out.println(s.age);  
        s.display();  
    }  
}
```

20
20

When you don't write a package statement, Java places the class in the default package (also called the unnamed package).

```
package college;
```

```
class Student {  
    int age = 20;  
  
    void display() {  
        System.out.println(age);  
    }  
}
```

```
package college;
```

```
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
  
        System.out.println(s.age); // ✓ Allowed  
        s.display();               // ✓ Allowed  
    }  
}
```

Both classes are in the college package, so access is still allowed.

```
package college;
```

```
public class Student {  
    int age = 20;  
  
    void display() {  
        System.out.println(age);  
    }  
}
```

```
package test;
```

```
import college.Student;
```

```
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
  
        System.out.println(s.age); // □ Error  
        s.display();               // □ Error  
    }  
}
```

Now Student and Main belong to **different packages**, so the default members (age and display()) cannot be accessed.

Declaration	Access Specifier	Accessible in Same Package?	Accessible in Different Package?
int age = 20;	Default (package-private)	✓ Yes	✗ No
private int age = 20;	Private	✗ No	✗ No
public int age = 20;	Public	✓ Yes	✓ Yes
protected int age = 20;	Protected	✓ Yes	✓ (only through inheritance in another package)

Protected Access Specifier

A protected member can be accessed:

- ✓ Within the same class
- ✓ Within the same package (even if not inherited)
- ✓ In a subclass (**derived class**), **even if it is in a different package**

```
class Animal {  
    protected void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```

}

class Dog extends Animal {
    void bark() {
        sound();
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.bark();
    }
}

```

Animal makes a sound

```
package college;
```

```
public class Student {
    protected int age = 20;
}

```

```
package test;
import college.Student;
```

```
public class Main {
    public static void main(String[] args) {
        Student s = new Student();

        System.out.println(s.age);
        // Compile-time Error. Main is not a subclass of Student
    }
}

```

```
package college;
```

```
public class Student {
    protected int age = 20;
}

```

```
package test;
import college.Student;
```

```
public class Main extends Student {

    public static void main(String[] args) {
        Main m = new Main();

        System.out.println(m.age); // ✓ Allowed
    }
}

```

If you want to access the public member variable from outside the package

<pre>package pack1; public class A { public int x = 100; }</pre>	<pre>package pack2; import pack1.A; public class B { public static void main(String[] args) { A obj = new A(); System.out.println(obj.x); // ✓ Accessible } }</pre>
Without import (using fully qualified class name)	<pre>package pack2; public class B { public static void main(String[] args) { pack1.A obj = new pack1.A(); System.out.println(obj.x); // ✓ Accessible } }</pre>

How to implement access specifier concepts practically

Create a folder P1 and place the program A.java inside the folder

```
package P1;
public class A
{
    public int a=10;
    public static void main(String[] s)
    {
    }
}
```

Create another folder P2 and place the program B.java inside the folder

```
package P2;
import P1.A;
public class B
{
    public static void main(String[] a)
    {
        A obj=new A();
        System.out.println(obj.a);
    }
}
```

Now compile and run the program

```
javac P1\A.java P2\B.java
```

```
java P2.B
```

```
Output: 10
```